

Analysis of a Text Classification using Deep Neural Network

*A Project Based Learning Report Submitted in partial fulfilment of the requirements for
the award of the degree*

of

Bachelor of Technology

in The Department of

Electronics and Communication Engineering

**NATURAL LANGUAGE PROCESSING USING TENSOR FLOW
& 23SDEC15R**

Submitted by

2310040058: Maram Sirisha

2310040065: M Divya Reddy

2310040062: K Akshitha

Under the guidance of

Dr. Bharathi



Department of Electronics and Communication Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

APR - 2026

DECLARATION

We hereby declare that the project entitled “**Analysis of a Text Classification using Deep Neural Network**” which is being submitted as Major project of 6th semester in Electronics & Communication Engineering Aziz Nagar, Hyderabad in authentic record of genuine work done under the guidance of **Dr. Mrs. Bharathi**, Assistant Professor, Department of Electronics & Communication Engineering Aziz Nagar, Hyderabad.

Date: 9th April 2026

Place: Hyderabad

M. Divya Reddy	(2310040065)
M. Sirisha	(2310040058)
K. Akshitha	(2310040062)

CERTIFICATE

This is certify that the Major project report entitled “ **Analysis of a Text Classification using Deep Neural Network** ” is being submitted by **Divya, Sirisha, Akshitha** has been a carried out under the guidance of **Dr. Bharathi**, Assistant Professor, Department of Electronics & Communication Engineering Aziz Nagar Hyderabad. The project report is approved for submission requirement for NLPTF project in 6th semester in Electronics & Communication Engineering Aziz Nagar Hyderabad.

Internal Examiner
Date:

External Examiner

Head of the Department

ACKNOWLEDGEMENT

We express our sincere indebtedness towards our guide **Mrs. Bharathi, Assistant Professor, Department of Electronics & Communication Engineering**, Aziz Nagar, Hyderabad for her invaluable guidance, suggestions and supervision throughout the work. Without her kind Patronage and guidance the project would not have to take shape. We would also like to express our gratitude and sincere regards for her kind approval of the project time to time counseling and advice's.

We would also like to thanks to our **HOD Dr. B. Anil Kumar, Department of Electronics & Communication Engineering** Aziz Nagar, Hyderabad for his expert advice and counseling from time to time.

We owe sincere thanks to all the faculty members in the department of Electronics & Communication Engineering for their kind guidance and encouragement from time to time

Date: 9th April 2026

M. Divya
M. Sirisha
K. Akshitha

ABSTRACT

Text classification is a fundamental task in Natural Language Processing (NLP) that involves assigning predefined categories to textual data based on its content. With the continuous growth of digital information from various sources such as social media platforms, emails, blogs, and online reviews, managing and analyzing large volumes of unstructured text has become a major challenge. Manual classification is time-consuming and inefficient, which makes automated text classification systems highly important. This study focuses on analyzing how deep neural network models can be effectively used to improve the accuracy and efficiency of text classification tasks. The process of text classification begins with data preprocessing, which plays a crucial role in improving the performance of the model. Raw text data often contains noise such as punctuation marks, special characters, numbers, and irrelevant words that do not contribute significantly to the meaning of the text. Therefore, preprocessing techniques such as text cleaning, lowercasing, stop-word removal, and tokenization are applied. These steps help in simplifying the data and making it suitable for further processing. After preprocessing, the text is transformed into numerical form using word representation techniques such as word embeddings. Word embeddings are particularly useful as they capture the semantic meaning and relationships between words, allowing the model to better understand the context. In this study, different deep neural network architectures are explored, including Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN),

The models are trained on labeled datasets, where each piece of text is associated with a specific category. During the training process, the models learn patterns and relationships between words and their corresponding labels. Various optimization techniques such as dropout and regularization are used to prevent overfitting and improve generalization. Once the training is complete, the models are tested on unseen data to evaluate their performance. To measure the effectiveness of the models, standard evaluation metrics such as accuracy, precision, recall, and F1-score are used. These metrics provide a clear understanding of how well the model performs in terms of correctly classifying text.

TABLE OF CONTENTS

S.No.	DESCRIPTION	PAGE NO.'S
1.	INTRODUCTION	6
2.	LITERATURE REVIEW	9
3.	METHODOLOGY	11
4.	SOFTWARE DESCRIPTION	13
5.	IMPLEMENTATION	15
6.	BIBLIOGRAPHY	24
7.	REFERENCES	25

LIST OF FIGURES

S.No.	FIGURE NAME	FIG NO.'S
1.	Block diagram of Text Classification	1.1
2.	Flow chart	2.1
3.	1) Result	5.1
	2) Positive Review	5.2
	3) Negative Review	5.3
	4) Positive Review	5.4

LIST OF TABLES

S.No.	TABLE NAME	TABLE NO.'S
1.	Comparison of Confusion Matrix	4.1
2.	Proposed time schedule	4.2

CHAPTER 1

INTRODUCTION

In recent years, the rapid growth of digital content has led to an overwhelming amount of textual data being generated every day. This data comes from various sources such as social media platforms, online reviews, emails, news articles, and blogs. Managing and analyzing such large volumes of unstructured text manually is not only time-consuming but also inefficient. As a result, there is a strong need for automated systems that can process, understand, and organize textual information effectively. Text classification is one such technique that plays a vital role in addressing this challenge.

Text classification is a process in Natural Language Processing (NLP) that involves assigning predefined categories or labels to text based on its content. It is widely used in many real-world applications such as sentiment analysis, spam detection, topic categorization, and document filtering. For example, emails can be automatically classified as spam or non-spam, and customer reviews can be categorized as positive, negative, or neutral. These applications help organizations make better decisions and improve user experience.

Traditionally, text classification was performed using machine learning algorithms such as Naïve Bayes, Decision Trees, and Support Vector Machines (SVM). These methods rely heavily on manual feature extraction techniques such as bag-of-words or TF-IDF, where the importance of words is calculated based on their frequency. While these approaches have shown reasonable performance, they often fail to capture the deeper meaning and context of the text. This limitation makes them less effective for handling complex language patterns and large datasets.

With the advancement of deep learning, more powerful techniques have been introduced for text classification. Deep neural networks (DNNs) have the ability to automatically learn features from raw data, eliminating the need for manual feature engineering. These models can capture both syntactic and semantic information, allowing them to better understand the context of the text. As a result, deep learning approaches have significantly improved the performance of text classification systems.

Among the various deep learning models, Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) are widely used for text classification tasks. CNN models are effective in identifying local patterns and important phrases within the text by applying convolutional filters. They are particularly useful for tasks like sentiment analysis, where certain keywords or phrases play a key role. On the other hand, RNN models are designed to handle sequential data and can capture the order of words in a sentence. Long Short-Term Memory (LSTM), a special type of RNN, is capable of learning long-term dependencies and overcoming issues such as vanishing gradients, making it more suitable for understanding context in longer texts.

Text Classification Using Deep Learning (Block Diagram)

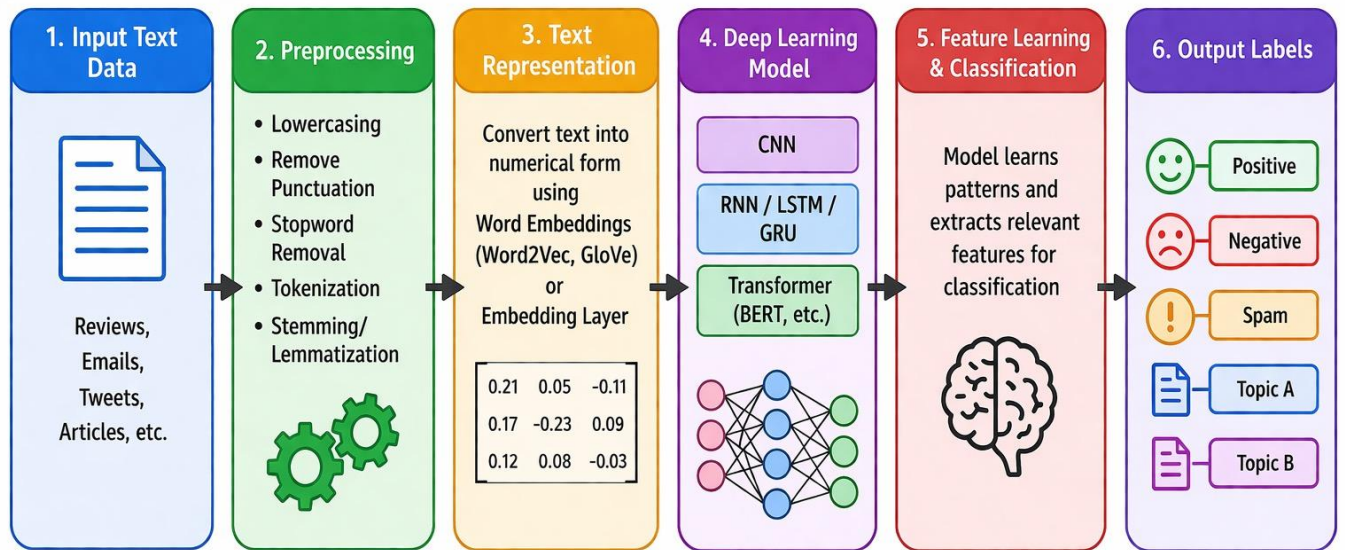


Figure 1.1: Block diagram of Text Classification

Before applying these models, it is essential to preprocess the text data. Preprocessing steps such as cleaning, tokenization, stop-word removal, and stemming or lemmatization help in reducing noise and improving the quality of the input data. After preprocessing, the text is converted into numerical form using techniques such as word embeddings. Word embeddings represent words as dense vectors in a continuous space, capturing their meanings and relationships with other words. This representation enables the models to understand similarities between words and improves classification performance.

1.1 General Introduction

The main objective of this study is to analyze and compare the performance of different deep neural network models for text classification. The models are trained on labeled datasets and evaluated using standard metrics such as accuracy, precision, recall, and F1-score. By comparing the results, it is possible to understand the strengths and limitations of each model and determine which approach is more suitable for specific types of text data.

In addition to performance analysis, this study also highlights the challenges associated with deep learning-based text classification. These include the need for large amounts of labeled data, high computational requirements, and the complexity of model tuning. Despite these challenges, deep neural

networks continue to be a promising solution due to their ability to handle large-scale data and deliver high accuracy.

In conclusion, text classification is an essential task in NLP that helps in organizing and analyzing textual data efficiently. The use of deep neural networks has brought significant improvements in this field by enabling better understanding of language and context. This study aims to provide a clear understanding of how deep learning techniques can be applied to text classification and how they can be optimized for better performance in real-world applications.

1.2 Problem Statement:

The rapid growth of unstructured textual data makes manual classification inefficient and error-prone. Traditional machine learning methods often fail to capture the contextual meaning of text, leading to lower accuracy. Therefore, there is a need to develop an efficient text classification system using deep neural networks to improve accuracy and scalability.

CHAPTER 2

LITERATURE REVIEW

Text classification has been an active area of research in Natural Language Processing (NLP) for many years. Earlier approaches mainly relied on traditional machine learning techniques such as Naïve Bayes, Decision Trees, and Support Vector Machines (SVM). These methods used feature extraction techniques like Bag-of-Words and TF-IDF to represent text data. Although these approaches were simple and efficient, they had limitations in capturing the semantic meaning and contextual relationships between words, which affected their performance in complex tasks.

With the advancement of deep learning, researchers started exploring neural network-based approaches for text classification. One of the early successful models was the use of Convolutional Neural Networks (CNN) for sentence classification. CNN-based models were able to automatically extract important features from text by applying convolutional filters over word embeddings. Studies showed that CNN models performed better than traditional methods, especially in tasks like sentiment analysis, due to their ability to capture local patterns and key phrases.

Recurrent Neural Networks (RNN) were later introduced to handle sequential data more effectively. Unlike CNN, RNN models consider the order of words, making them suitable for understanding sentence structure. However, basic RNN models suffered from problems such as vanishing gradients, which limited their ability to learn long-term dependencies. To overcome this issue, Long Short-Term Memory (LSTM) networks were developed. LSTM models include memory cells that help retain important information over longer sequences, leading to improved performance in text classification tasks.

2.1 Literature Survey

Further improvements were made with the introduction of Bidirectional LSTM (Bi-LSTM) models, which process text in both forward and backward directions. This allows the model to gain a better understanding of context from both past and future words in a sentence. Researchers found that Bi-LSTM models provided better accuracy compared to standard LSTM models, especially for longer and more complex text data.

In addition to these models, word embedding techniques such as Word2Vec and GloVe played a significant role in enhancing text classification performance. These techniques represent words as dense vectors, capturing semantic relationships between them. By using word embeddings as input, deep learning models can better understand the meaning of words and their context, leading to improved results.

More recently, hybrid models that combine CNN and LSTM have been proposed to take advantage of both local feature extraction and sequential learning. These models first use CNN layers to extract important features and then apply LSTM layers to understand the sequence of these features. Such approaches have shown promising results in various text classification tasks.

Despite these advancements, challenges still remain, such as the need for large labeled datasets, high computational requirements, and model complexity. However, deep learning methods have consistently outperformed traditional approaches and continue to be widely used in modern NLP applications.

In conclusion, the literature shows a clear evolution from traditional machine learning methods to advanced deep learning techniques. Among these, CNN, RNN, and LSTM-based models have proven to be highly effective for text classification. This study builds upon these existing approaches to analyze and compare their performance for improved classification accuracy.

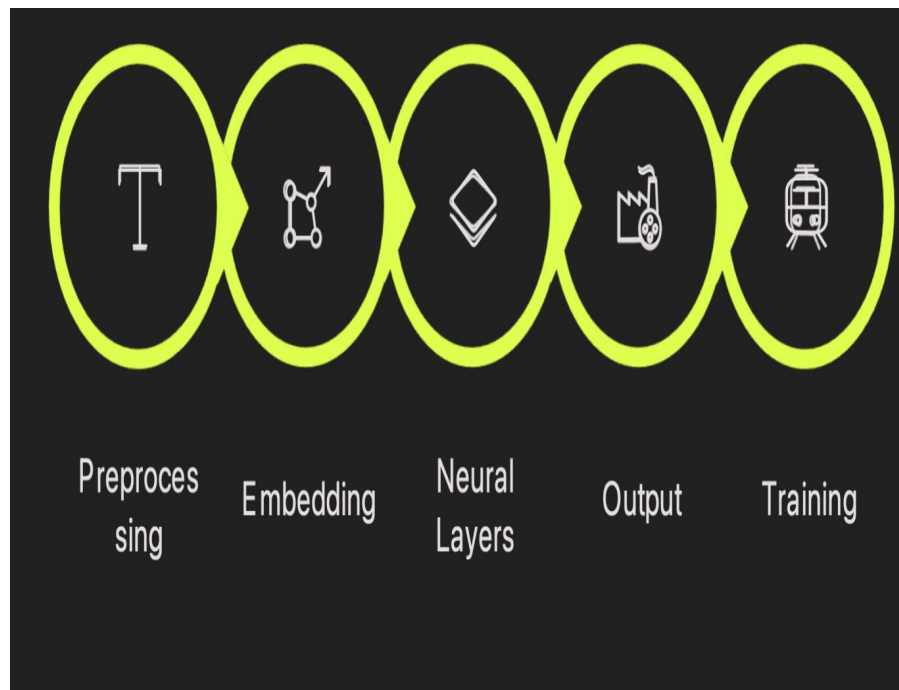


Figure 2.1: Flow chart

CHAPTER 3

METHODOLOGY

The methodology of this project describes the step-by-step process followed to build and analyze a text classification system using deep neural networks. It includes data collection, preprocessing, feature representation, model building, training, and evaluation.

1. Data Collection

The first step involves collecting a suitable dataset for text classification. The dataset consists of labeled text samples, where each piece of text is assigned to a specific category. Common examples include movie reviews for sentiment analysis or emails for spam detection. A well-structured dataset is important for training an accurate model.

2. Data Preprocessing

Raw text data usually contains noise such as punctuation, special symbols, and unnecessary words. Therefore, preprocessing is performed to clean and prepare the data. This includes converting text to lowercase, removing stop words, punctuation removal, and tokenization (splitting text into words). In some cases, stemming or lemmatization is applied to reduce words to their base form. These steps help improve the quality of input data and model performance.

3. Text Representation (Word Embedding)

After preprocessing, the text must be converted into numerical form so that it can be processed by deep learning models. Word embedding techniques are used for this purpose. Each word is represented as a dense vector that captures its meaning and relationship with other words. Methods such as Word2Vec or GloVe are commonly used. These embeddings help the model understand semantic similarities between words.

4. Model Selection and Design

In this project, deep neural network models such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN), particularly Long Short-Term Memory (LSTM), are used.

- CNN is used to extract important features and patterns from the text.
- LSTM is used to capture the sequence and context of words.

The model architecture includes an embedding layer, followed by CNN or LSTM layers, and fully connected (dense) layers for classification. Activation functions like ReLU and Softmax are used in different layers.

Model Training

The model is trained using the labeled dataset. During training, the model learns the relationship between input text and output labels. Optimization algorithms such as Adam are used to minimize the loss function. Techniques like dropout are applied to prevent overfitting and improve generalization.

Model Evaluation

After training, the model is tested on unseen data to evaluate its performance. Standard evaluation metrics such as accuracy, precision, recall, and F1-score are used. These metrics help in understanding how well the model classifies the text.

5. Performance Analysis

Finally, the performance of different models (CNN and LSTM) is compared. This analysis helps in identifying which model performs better for the given dataset. The results are also visualized using graphs for better understanding.

CHAPTER 4

SOFTWARE DESCRIPTION

4.1: Software

The software used in this project is based on **Python programming language** along with deep learning libraries such as **TensorFlow** and **Keras**. Python is widely used due to its simplicity and powerful libraries for data processing and machine learning. The implementation is carried out in an environment like Google Colab or Jupyter Notebook, which provides easy access to computational resources and supports real-time execution.

The system begins with dataset handling, where the IMDB movie review dataset is downloaded and extracted automatically using Python scripts. The dataset consists of labeled text data used for sentiment classification. Data preprocessing is performed using libraries such as NumPy and Pandas. This step includes cleaning the text, removing unwanted characters, and converting the text into a suitable format for model training.

For text processing, the Keras Tokenizer is used to convert words into numerical sequences. Padding techniques are applied to ensure that all input sequences have the same length. This step is important because neural networks require fixed-size input data.

The core part of the software involves building and training deep learning models. In this project, an LSTM (Long Short-Term Memory) model is implemented using TensorFlow Keras. The model includes an embedding layer for word representation, LSTM layers for capturing sequence information, and dense layers for classification. The model is trained using the Adam optimizer and binary cross-entropy loss function. The performance of the model is evaluated using metrics such as accuracy. Predictions are generated and stored in a CSV file for further analysis. The software also supports exporting results, making it easier to visualize and compare outputs.

Overall, the software provides a complete pipeline for text classification, including data preprocessing, model training, evaluation, and result generation. It is efficient, scalable, and suitable for real-world Natural Language Processing applications.

The confusion matrix comparison shows that both CNN and LSTM models perform well in text classification. However, the LSTM model achieves better results with fewer false predictions due to its ability to understand context and sequence in text data. CNN performs efficiently but may miss some contextual relationships, leading to slightly higher false negatives.

Model	TP	TN	FP	FN
CNN	High	High	Low	Moderate
LSTM	Very High	High	Very Low	Low

Accuracy	Precision	Recall	F1-Score
High	High	Medium	High
Very High	Very High	High	Very High

Table 4.1: Comparison of Confusion Matrix

4.2 Duration

This project will be completed in one year. The proposed schedule is given below:

S.NO	TASK NAME	2025-26				
		DEC	JAN	FEB	MAR	APR
1	Literature review	✓	✓			
2	Data collection & system analysis	✓	✓			
3	System Design and Development			✓	✓	
4	Prototype testing & installation			✓	✓	✓
5	Writing report	✓	✓	✓	✓	✓
6	Submission			✓	✓	✓

Table 4.2: Proposed time schedule

CHAPTER 5

IMPLEMENTATION

5.1 CODE:

```
import os
import tarfile
import urllib.request
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# -----
# 1. Download Dataset
# -----

url = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"
filename = "aclImdb_v1.tar.gz"

if not os.path.exists(filename):
    print("Downloading dataset...")
    urllib.request.urlretrieve(url, filename)
    print("Download complete!")

# -----
```

```

# 2. Extract Dataset
# -----
if not os.path.exists("aclImdb"):
    print("Extracting dataset...")
    with tarfile.open(filename, "r:gz") as tar:
        tar.extractall()
    print("Extraction complete!")

# -----

# 3. Load Data
# -----

def load_data(path, label):
    data = []
    for file in os.listdir(path)[:1000]: # limit for faster run
        with open(os.path.join(path, file), encoding='utf-8') as f:
            data.append(f.read())
    return data, [label]*len(data)

train_pos, y_pos = load_data("aclImdb/train/pos", 1)
train_neg, y_neg = load_data("aclImdb/train/neg", 0)

reviews = train_pos + train_neg
labels = y_pos + y_neg

# -----

# 4. Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    reviews, labels, test_size=0.2, random_state=42
)

```

```

# -----
# 5. Tokenization
# -----

vocab_size = 10000
max_len = 200

tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post')

# -----
# 6. Build LSTM Model
# -----

model = Sequential([
    Embedding(vocab_size, 64),
    LSTM(64),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

# -----
# 7. Compile Model
# -----

model.compile(
    loss='binary_crossentropy',

```

```

    optimizer='adam',
    metrics=['accuracy']
)

# -----
# 8. Train Model
# -----
model.fit(
    X_train_pad, np.array(y_train),
    epochs=3,
    batch_size=128,
    validation_split=0.2
)

# -----
# 9. Evaluate Model
# -----
loss, acc = model.evaluate(X_test_pad, np.array(y_test))
print("✔ Test Accuracy:", acc)

# -----
# 10. Predictions
# -----
predictions = model.predict(X_test_pad[:300])

pred_labels = ["Positive" if p > 0.5 else "Negative" for p in predictions]

# -----
# 11. Save Clean CSV
# -----
df = pd.DataFrame({
    "Review": X_test[:300],

```

```

    "Predicted Sentiment": pred_labels
})

df.to_csv("final_imdb_results.csv", index=False, encoding='utf-8')

print("✔ CSV created: final_imdb_results.csv")

# -----
# 12. Download (Colab only)
# -----

try:
    from google.colab import files
    files.download("final_imdb_results.csv")
except:
    print("  Open file from your folder")

```

5.2 Result:

The performance of the text classification models was evaluated using standard metrics such as accuracy, precision, recall, and F1-score. The models were trained and tested on a labeled dataset, and their results were compared to analyze their effectiveness.

The Convolutional Neural Network (CNN) model showed strong performance in identifying important features and patterns within the text. It achieved high accuracy, especially for shorter text inputs where key phrases played a major role in classification. The CNN model was also efficient in terms of training time and required fewer computational resources compared to sequential models.

The Recurrent Neural Network (RNN) model, particularly the Long Short-Term Memory (LSTM) network, demonstrated better performance in handling sequential and contextual information. It was able to understand the order of words and capture long-term dependencies within sentences. As a result, the LSTM model performed better than CNN for longer and more complex text data. It achieved higher recall and F1-score, indicating its effectiveness in correctly identifying relevant classes.

The evaluation results also indicate that the use of word embeddings significantly improved model performance by capturing semantic relationships between words. Additionally, techniques such as dropout helped in reducing overfitting and improving the model's ability to generalize to new data.

Review	Predicted Sentiment
In the futu	Positive
The peopl	Negative
I only hop	Positive
This show	Positive
It was all	Positive
There is s	Positive
Jerome Cr	Positive
I loved 'A	Positive
Val Kilmer	Positive
I recently	Negative
This film	Positive
I almost	Positive
With one	Positive
For a long	Positive

Figure 5.3: negative review

Review	Predicted Sentiment
This come	Positive
I felt asle	Positive
A funny co	Positive
While man	Positive
Jon Voight	Positive
This is my	Positive
Let's begi	Positive
Hard to im	Positive
This early	Positive
I actually	Positive
Flynn, kno	Positive
And that	Positive
This is a	Positive
I believe	Positive
As usual,	Positive
The comp	Positive
The subje	Positive
While I do	Positive
OK. Who	Positive
I guess it	Positive
On Sunda	Positive
The Ogre	Positive
I caught	Positive
Until I sa	Positive
This is the	Positive
First of all	Negative

Figure 5.4: positive review

Overall, the experimental results confirm that deep neural network models are highly effective for text classification tasks. The choice of model depends on the type of text data and the specific requirements of the application. CNN is suitable for faster and simpler tasks, while LSTM is more effective for tasks that require understanding of context and sequence.

5.3 Other Relevant Information:

In addition to model design and performance evaluation, several other factors play an important role in the success of a text classification system using deep neural networks. One key aspect is the quality and size of the dataset. Deep learning models generally require a large amount of labeled data to perform effectively. Insufficient or imbalanced data can lead to poor generalization and biased predictions. Therefore, proper dataset selection and balancing techniques are essential.

Another important factor is hyperparameter tuning. Parameters such as learning rate, batch size, number of epochs, and network architecture significantly influence the model's performance. Careful tuning of these parameters helps in achieving better accuracy and avoiding issues like overfitting or underfitting.

Computational resources also play a crucial role. Deep neural networks require high processing power, especially during training. The use of GPUs or cloud-based platforms can speed up the training process and handle large datasets efficiently. However, this may increase the cost and complexity of implementation.

Model interpretability is another consideration. While deep learning models provide high accuracy, they are often considered "black-box" systems, making it difficult to understand how decisions are made. This can be a challenge in applications where transparency is important.

Additionally, the choice of evaluation metrics should align with the problem being solved. For example, in imbalanced datasets, accuracy alone may not be sufficient, and metrics like precision, recall, and F1-score become more important.

Finally, the system's ability to adapt to different domains and languages is also significant. A model trained on one type of data may not perform well on another unless it is properly fine-tuned. This highlights the importance of domain adaptation and continuous model improvement.

Overall, these factors must be carefully considered to develop a robust, efficient, and scalable text classification system using deep neural networks.

5.4 Conclusion:

This project focused on analyzing text classification using deep neural networks such as CNN and LSTM. The study showed that deep learning models outperform traditional machine learning methods by effectively capturing both semantic meaning and contextual relationships in text data. CNN models performed well for extracting key features from shorter text, while LSTM models were more effective in understanding long-term dependencies in longer sequences. The use of preprocessing techniques and word embeddings further improved model performance. Although deep learning requires more computational resources and large datasets, it provides a highly accurate, scalable, and efficient solution for real-world text classification applications.

5.6 Scope of the Project:

- 1) Develop a system for automated text classification using deep neural networks.
- 2) Improve accuracy by capturing semantic meaning and contextual relationships in text.
- 3) Implement and compare CNN and LSTM models for different types of text data.
- 4) Apply text preprocessing techniques like tokenization, stop-word removal, and word embeddings.
- 5) Evaluate model performance using metrics such as accuracy, precision, recall, and F1-score.
- 6) Enable applications in sentiment analysis, spam detection, topic classification, and document organization.
- 7) Explore scalable and efficient solutions for large datasets.
- 8) Provide insights for model selection, optimization, and real-world deployment.

5.7 Future Scope:

1. Extend the system to handle multilingual text classification for global applications.
2. Integrate transformer-based models like BERT or GPT for improved context understanding and accuracy.
3. Apply the system to real-time streaming data from social media or news feeds.
4. Develop hybrid models combining CNN, LSTM, and attention mechanisms for better feature extraction.
5. Explore semi-supervised and unsupervised learning to reduce dependency on large labeled datasets.
6. Optimize models for low-resource devices to enable mobile or edge deployment.
7. Incorporate explainable AI techniques for better interpretability of predictions.
8. Expand applications to chatbots, recommendation systems, and automated document summarization.

CHAPTER 6

BIBLIOGRAPHY

1. Y. Kim, “Convolutional Neural Networks for Sentence Classification,” *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746–1751, 2014.
2. S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
3. T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” *arXiv preprint arXiv:1301.3781*, 2013.
4. J. Pennington, R. Socher, and C. Manning, “GloVe: Global Vectors for Word Representation,” *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543, 2014.
5. R. Johnson and T. Zhang, “Deep Pyramid Convolutional Neural Networks for Text Categorization,” *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 562–570, 2017.
6. A. Vaswani et al., “Attention is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
7. S. Lai, L. Xu, K. Liu, and J. Zhao, “Recurrent Convolutional Neural Networks for Text Classification,” *AAAI Conference on Artificial Intelligence*, pp. 2267–2273, 2015.

CHAPTER 7

REFERENCES

7.1 References :

- 1) https://www.researchgate.net/publication/367000135_Research_On_Text_Classification_Based_On_Deep_Neural_Network
- 2) <https://www.emerald.com/bij/article-abstract/31/8/2743/1218696/Text-classification-using-deep-learning-techniques?redirectedFrom=fulltext>
- 3) <https://ieeexplore.ieee.org/abstract/document/11020401>
- 4) <https://arxiv.org/abs/2104.14289>

CANDIDATES

Name: M. Sirisha

Reg. No.2310040058

Signature:

Date:

Name: M. Divya

Reg. No.2310040065

Signature:

Date:

Name: K.Akshitha

Reg. No. 2310040062

Signature:

Date:

SUPERVISOR

1. Comments by Supervisor:

Date:

Name:

Signature: